

Autograd как естественная эволюция вычисления градиентов

Содержание

1	Введение: зачем вообще нужен autograd	3
1.1	Как autograd проявляется в коде PyTorch	3
2	Наивный подход: градиентный спуск вручную	4
2.1	Пример ручного обучения одного веса	4
2.2	Почему этот подход быстро перестаёт быть удобным	5
2.3	Иллюстрация: как градиентный спуск двигает вес	5
3	Промежуточный шаг: символьное дифференцирование	5
3.1	Чем символьное дифференцирование отличается от ручного подхода	6
3.2	Почему символьная алгебра - это не autograd	6
3.3	Почему символьный подход неудобен для глубокого обучения	7
3.4	Наглядное сравнение	7
3.5	Вывод	8
4	Обратное распространение ошибки: идея, которая делает всё масштабируемым	8
4.1	Как это работает на конкретном примере	8
4.2	Почему здесь появляется слово “ошибка”	9
4.3	Тот же пример в виде кода	10
4.4	Визуальная идея backpropagation	11
4.5	Ограничение ручного backpropagation	11
5	Матричный подход: почему линейная алгебра так важна	11
5.1	Линейный слой для целого батча	11
5.2	Градиенты в матричной форме	12
5.3	Пример: ручное обучение в матричной форме без autograd	12
5.4	Почему матричный подход эффективен	13
6	Autograd в PyTorch: тот же backpropagation, но уже автоматически	14
6.1	Тот же матричный пример, но уже с autograd	14
6.2	Что здесь делает autograd	15
6.3	Две детали, которые почти всегда всплывают в начале	15
7	Нормальный рабочий вариант: torch.optim	15
7.1	Пример небольшой модели с Adam	16

1. Введение: зачем вообще нужен autograd

Обучение нейронной сети - это задача подбора параметров модели так, чтобы ошибка на данных была как можно меньше. Формально обычно пишут

$$\min_W L(f(x, W), y),$$

где x - входные данные, W - веса модели, f - сама модель, а L - функция потерь. Чтобы уменьшать L , нам нужно понимать, как малое изменение каждого параметра влияет на итоговую ошибку. Именно для этого и нужны градиенты:

$$\frac{\partial L}{\partial W}.$$

На совсем маленьких примерах производные можно выписать вручную. Но уже в сети из нескольких слоёв функция потерь превращается в длинную композицию операций: матричных умножений, сложений, нелинейностей, нормализаций и так далее. Для каждой новой архитектуры заново выводить все производные неудобно, а для глубоких моделей - практически нереально.

Идея **autograd** состоит в том, чтобы не просить разработчика руками дифференцировать большую формулу целиком. Вместо этого система:

1. строит вычислительный граф во время прямого прохода;
2. запоминает, какие элементарные операции были выполнены;
3. затем автоматически применяет правило цепочки в обратном направлении.

1.1. Как autograd проявляется в коде PyTorch

В PyTorch autograd включается там, где тензоры объявляются с флагом `requires_grad=True`. После этого все операции над такими тензорами начинают строить вычислительный граф.

```
1 import torch
2
3 x = torch.tensor([[2.0]])
4 w = torch.tensor([[1.0]], requires_grad=True)
5 b = torch.tensor([0.0], requires_grad=True)
6 y_true = torch.tensor([[4.0]])
7
8 y_pred = x @ w + b
9 loss = ((y_pred - y_true) ** 2).mean()
10
11 print(loss.grad_fn)
12 loss.backward()
13
14 print(w.grad)
15 print(b.grad)
```

Коротко: в коде PyTorch autograd виден в трёх местах.

- `requires_grad=True` - говорим, что по этому тензору нужно считать производные;
- `loss.backward()` - запускаем обратный проход;
- `parameter.grad` - читаем уже вычисленные градиенты.

Важно также понимать, что autograd сам по себе **не обновляет параметры**. Он только вычисляет градиенты. Обновление весов - это уже отдельный шаг, который можно сделать вручную или через `torch.optim`.

2. Наивный подход: градиентный спуск вручную

Начнём с самой простой модели: один вход, один вес и квадратичная ошибка. Пусть модель имеет вид

$$\hat{y} = wx, \quad L = (\hat{y} - y)^2.$$

В этом случае производную можно посчитать руками:

$$\frac{dL}{dw} = 2(\hat{y} - y)x = 2(wx - y)x.$$

После этого остаётся применить обычный градиентный спуск:

$$w \leftarrow w - \eta \frac{dL}{dw},$$

где η - шаг обучения.

Этот подход полезен как отправная точка: он показывает, что обучение нейросети - это не что-то мистическое, а обычная численная оптимизация. Мы вычисляем ошибку, вычисляем производную, немного сдвигаем параметр в сторону уменьшения ошибки и повторяем процесс.

2.1. Пример ручного обучения одного веса

```
1 x = 2.0
2 y_true = 4.0
3
4 w = 0.5
5 lr = 0.1
6
7 for step in range(8):
8     y_pred = w * x
9     loss = (y_pred - y_true) ** 2
10
11     grad_w = 2 * (y_pred - y_true) * x
12     w = w - lr * grad_w
13
14     print(f"step={step:2d} w={w:.4f} loss={loss:.6f}")
```

Здесь всё прозрачно: формула производной записана руками, и потому каждая строчка кода понятна. Но именно в этом и ограничение. Стоит добавить ещё один слой, ещё один параметр или нелинейность, и ручной вывод производных быстро становится громоздким.

2.2. Почему этот подход быстро перестаёт быть удобным

Для одного веса формула градиента получается в одну строку. Для десятков тысяч весов и нескольких слоёв ситуация уже совсем другая:

- нужно вручную выводить много производных;
- легко ошибиться в одном знаке или множителе;
- формулы надо переписывать при каждом изменении модели;
- код перестаёт быть читаемым.

То есть наивный подход хорош как учебный старт, но не как рабочий инструмент для глубоких сетей.

2.3. Иллюстрация: как градиентный спуск двигает вес

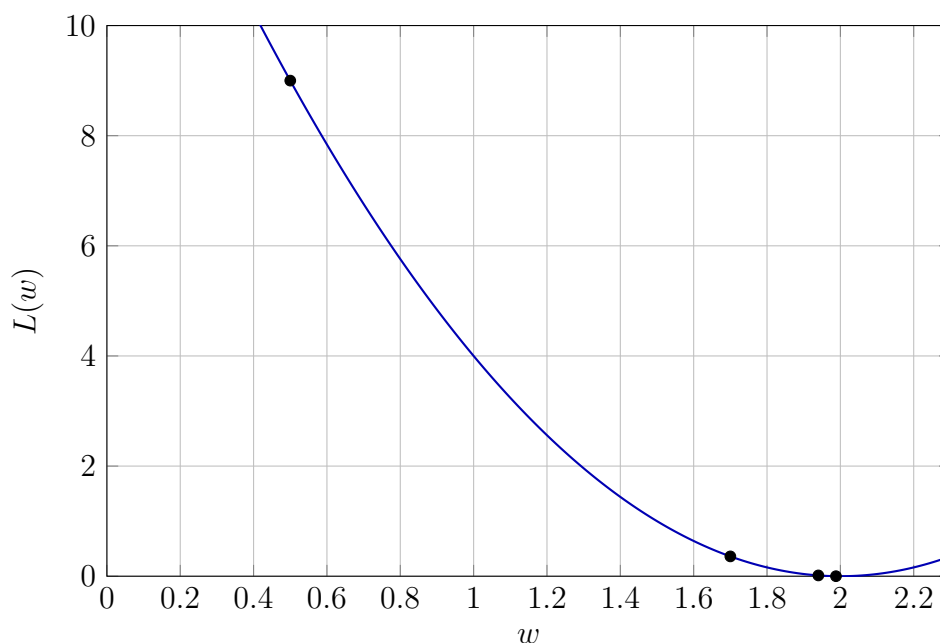


Рис. 1: Функция потерь для модели $\hat{y} = wx$ при $x = 2$ и $y = 4$. Точки показывают несколько шагов градиентного спуска.

3. Промежуточный шаг: символьное дифференцирование

Между полностью ручным вычислением производных и современным autograd есть ещё один важный подход - **символьное дифференцирование**. Его идея состоит в том, что формула рассматривается не как набор численных операций, а как **символьное выражение**, обычно представленное в виде дерева.

Например, если функция задана как

$$f(x) = (x^2 + 3x + 1)^2,$$

то система символьной алгебры хранит не численное значение этой функции в точке, а именно её структуру: узлы сложения, умножения, возведения в степень и переменную x . После этого к дереву можно применять известные правила дифференцирования и получать новую формулу, которая уже является аналитическим выражением для производной.

Для приведённого примера символьная система выведет

$$f'(x) = 2(x^2 + 3x + 1)(2x + 3).$$

3.1. Чем символьное дифференцирование отличается от ручного подхода

По сравнению с полностью ручным выводом производных символьный подход уже выглядит большим шагом вперёд. Человеку не нужно каждый раз самостоятельно раскрывать правило цепочки, правило произведения и другие свойства производных. Достаточно описать исходную формулу, а система сама построит формулу для производной.

Именно поэтому символьная алгебра исторически выглядит как естественный промежуточный этап:

- ручной подход - человек сам выводит формулу градиента;
- символьный подход - программа выводит аналитическую формулу градиента;
- autograd - программа не строит формулу целиком, а вычисляет производную по графу операций.

3.2. Почему символьная алгебра - это не autograd

На первый взгляд может показаться, что символьное дифференцирование и autograd делают одно и то же. В обоих случаях используется правило цепочки, в обоих случаях есть дерево или граф операций. Но принцип работы у них разный.

Символьное дифференцирование строит *новое аналитическое выражение* для производной. Результатом работы является формула.

Autograd обычно не пытается получить красивую формулу целиком. Вместо этого он выполняет исходные вычисления численно, строит вычислительный граф, а затем в обратном проходе вычисляет *численные значения производных* в конкретной точке.

То есть различие можно сформулировать так:

- символьная алгебра отвечает на вопрос: “какая формула у производной?”
- autograd отвечает на вопрос: “чему равна производная в этой конкретной точке вычислений?”

3.3. Почему символьный подход неудобен для глубокого обучения

Несмотря на свою математическую красоту, символьное дифференцирование плохо подходит для реального обучения больших нейронных сетей.

Во-первых, при последовательном применении правил дифференцирования формулы быстро разрастаются. Это явление часто называют **expression swell**: выражение для производной может оказаться значительно больше и сложнее, чем исходная функция. Даже если математически оно корректно, работать с ним становится тяжело.

Во-вторых, символьной системе обычно нужны дополнительные этапы упрощения выражений. Без этого производная может быть записана в громоздком и неэффективном виде. Но автоматическое упрощение больших формул - сама по себе сложная задача.

В-третьих, нейросети работают не с отдельными скалярными формулами, а с большими тензорными вычислениями: матрицами, батчами, слоями, операциями на GPU. В таких условиях нам почти никогда не нужна красивая аналитическая формула для производной. Нам нужно быстро и надёжно получить её численное значение для текущего батча и текущих параметров.

Наконец, современный код моделей часто содержит циклы, ветвления, условные конструкции и динамически меняющийся граф вычислений. Для символьной алгебры такие конструкции неудобны, тогда как autograd естественно работает именно с реальной последовательностью выполненных операций.

3.4. Наглядное сравнение

Характеристика	Символьное дифференцирование	Autograd
Что является результатом	Формула производной	Численное значение производной
Когда строится производная	Как новое выражение над исходной формулой	Во время обратного прохода по вычислительному графу
Нужно ли упрощать выражения	Часто да	Обычно нет
Хорошо ли подходит для больших тензоров и GPU	Ограниченно	Да
Хорошо ли работает с динамическим графом	Скорее плохо	Да
Типичный сценарий	CAS, аналитические пакеты, вывод формул	Обучение нейросетей и численная оптимизация

Символьная алгебра хочет получить формулу. Autograd хочет быстро получить число.

3.5. Вывод

Символьное дифференцирование - важный и очень полезный подход, но у него другая цель. Оно стремится получить **аналитическую формулу** производной. В задачах глубокого обучения этого обычно не требуется. Нам важнее быстро вычислять градиенты для конкретного прохода по данным, не раздувая формулы и не занимаясь их упрощением. Именно поэтому в машинном обучении центральное место заняли backpropagation и autograd, а не классические системы символьной алгебры.

4. Обратное распространение ошибки: идея, которая делает всё масштабируемым

Следующий шаг в эволюции - понять, что большую производную не обязательно выводить сразу целиком. Намного удобнее разложить вычисление на последовательность простых операций и применять правило цепочки.

Рассмотрим маленькую двухслойную модель:

$$h = w_1x, \quad \hat{y} = w_2h, \quad L = (\hat{y} - y)^2.$$

Здесь итоговая функция потерь зависит от w_1 не напрямую, а через промежуточную переменную h и затем через предсказание $\hat{y}$. Поэтому производная по w_1 считается как произведение локальных производных:

$$\frac{dL}{dw_1} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dh} \cdot \frac{dh}{dw_1}.$$

Именно эта идея и лежит в основе backpropagation: сначала мы считаем значения на прямом проходе, затем идём по вычислениям назад и на каждом шаге домножаем на локальную производную.

4.1. Как это работает на конкретном примере

Чтобы не оставлять формулу в абстрактном виде, разберём один полный проход с конкретными числами. Пусть

$$x = 2, \quad y = 4, \quad w_1 = 0.5, \quad w_2 = 1.5.$$

Тогда прямой проход выглядит так:

$$h = w_1x = 0.5 \cdot 2 = 1,$$

$$\hat{y} = w_2h = 1.5 \cdot 1 = 1.5,$$

$$L = (\hat{y} - y)^2 = (1.5 - 4)^2 = 6.25.$$

На этом шаге сеть ошиблась: предсказание равно 1.5, а правильный ответ равен 4. Теперь нужно понять, как эта ошибка связана с каждым параметром.

Сначала берём производную функции потерь по предсказанию:

$$\frac{dL}{d\hat{y}} = 2(\hat{y} - y) = 2(1.5 - 4) = -5.$$

Это и есть первый сигнал ошибки. Он показывает, в какую сторону надо менять предсказание $\hat{y}$, чтобы уменьшить функцию потерь. Отрицательный знак означает, что текущее предсказание слишком мало и его нужно увеличивать.

Теперь передадим этот сигнал назад к предыдущему узлу:

$$\frac{d\hat{y}}{dh} = w_2 = 1.5,$$

$$\frac{dL}{dh} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dh} = -5 \cdot 1.5 = -7.5.$$

Мы получили, как изменение скрытой переменной h влияет на итоговую ошибку. После этого можно вычислить градиент по первому весу:

$$\frac{dh}{dw_1} = x = 2,$$

$$\frac{dL}{dw_1} = \frac{dL}{dh} \cdot \frac{dh}{dw_1} = -7.5 \cdot 2 = -15.$$

А для второго веса производная считается ещё проще:

$$\frac{dL}{dw_2} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dw_2} = -5 \cdot h = -5 \cdot 1 = -5.$$

Итак, мы получили два градиента:

$$\frac{dL}{dw_1} = -15, \quad \frac{dL}{dw_2} = -5.$$

Если теперь сделать шаг градиентного спуска

$$w \leftarrow w - \eta \frac{dL}{dw},$$

то оба веса увеличатся, потому что градиенты отрицательны. Это логично: модель недооценила ответ, значит нужно сделать её выход больше.

Backpropagation можно воспринимать как передачу сигнала ошибки справа налево. Сначала мы понимаем, насколько ошиблось итоговое предсказание, потом пересчитываем этот сигнал для предыдущих промежуточных переменных и только после этого получаем градиенты по параметрам.

4.2. Почему здесь появляется слово “ошибка”

Теперь можно точнее понять, почему метод называется именно обратным распространением ошибки. На последнем слое мы вычисляем, насколько сильно предсказание отличается от правильного ответа. В нашем примере этот первичный сигнал равен

$$\frac{dL}{d\hat{y}} = -5.$$

Дальше он не исчезает, а передаётся назад через все промежуточные вычисления. Каждый предыдущий узел получает свою версию этого сигнала:

$$\frac{dL}{d\hat{y}} \rightarrow \frac{dL}{dh} \rightarrow \frac{dL}{dw_1}, \frac{dL}{dw_2}.$$

Именно поэтому говорят, что ошибка распространяется назад по сети. На каждом шаге она не просто копируется, а пересчитывается с помощью локальной производной.

4.3. Тот же пример в виде кода

Ниже записан тот же самый пример, но уже в виде программы. Здесь хорошо видно соответствие между математическими формулами и реальными вычислениями.

```

1 x = 2.0
2 y_true = 4.0
3
4 w1 = 0.5
5 w2 = 1.5
6 lr = 0.05
7
8 # forward
9 h = w1 * x
10 y_pred = w2 * h
11 loss = (y_pred - y_true) ** 2
12
13 print("FORWARD:")
14 print(f"h = {h:.4f}")
15 print(f"y_pred = {y_pred:.4f}")
16 print(f"loss = {loss:.4f}")
17
18 # backward
19 dL_dy = 2 * (y_pred - y_true)
20 dL_dw2 = dL_dy * h
21
22 dL_dh = dL_dy * w2
23 dL_dw1 = dL_dh * x
24
25 print("\nBACKWARD:")
26 print(f"dL/dy_pred = {dL_dy:.4f}")
27 print(f"dL/dh = {dL_dh:.4f}")
28 print(f"dL/dw1 = {dL_dw1:.4f}")
29 print(f"dL/dw2 = {dL_dw2:.4f}")
30
31 # update
32 w1 = w1 - lr * dL_dw1
33 w2 = w2 - lr * dL_dw2
34
35 print("\nUPDATE:")
36 print(f"new w1 = {w1:.4f}")
37 print(f"new w2 = {w2:.4f}")

```

Этот код полезен тем, что связывает сразу три уровня описания:

- формулу по правилу цепочки;
- численный пример с конкретными значениями;
- реальную реализацию backward в программе.

4.4. Визуальная идея backpropagation

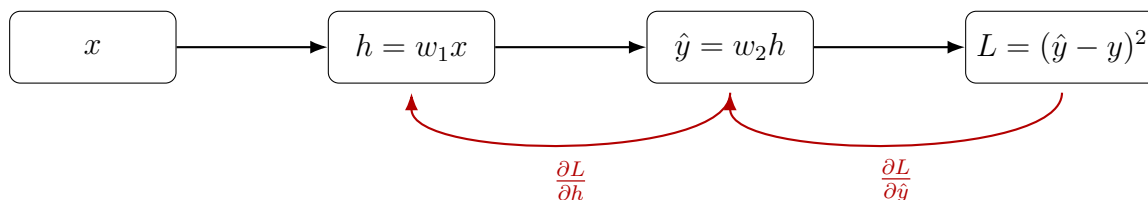


Рис. 2: Прямой проход идёт слева направо, а градиент - справа налево.

4.5. Ограничение ручного backpropagation

Хотя backpropagation решает проблему масштаба лучше, чем наивный способ, писать такой код руками для реальных сетей всё ещё неудобно. Разработчик по-прежнему обязан:

- помнить формулы для каждой операции;
- хранить промежуточные значения;
- аккуратно реализовывать обратный проход для каждого слоя.

Отсюда и возникает следующий шаг: оставить математическую идею backpropagation, но автоматизировать её в программной системе. Однако прежде чем перейти к полной автоматизации, важно понять ещё один уровень - матричную запись вычислений.

5. Матричный подход: почему линейная алгебра так важна

До сих пор все примеры были скалярными: один вход, один вес, одна ошибка. Но реальные нейросети почти никогда не работают со скалярами по одному. Они обрабатывают **векторы**, **матрицы** и целые **мини-батчи** объектов сразу.

Именно здесь появляется следующий ключевой шаг эволюции: переход от покомпонентных вычислений к матричной форме.

5.1. Линейный слой для целого батча

Пусть у нас есть матрица объектов

$$X \in \mathbb{R}^{n \times d},$$

где n - число объектов в батче, а d - число признаков. Тогда линейный слой можно записать так:

$$\hat{Y} = XW + b,$$

где

$$W \in \mathbb{R}^{d \times k}, \quad b \in \mathbb{R}^{1 \times k}, \quad \hat{Y} \in \mathbb{R}^{n \times k}.$$

Вместо того чтобы считать выход для каждого объекта и для каждого нейрона отдельно, мы выполняем одно матричное умножение и сразу получаем все предсказания батча.

$$\boxed{X \in \mathbb{R}^{n \times d}} \quad \times \quad \boxed{W \in \mathbb{R}^{d \times k}} \quad + \quad \boxed{b \in \mathbb{R}^{1 \times k}} \quad =$$

Рис. 3: Матричная запись линейного слоя. Одной формулой обрабатывается сразу весь мини-батч.

Замечание. Матричный подход и autograd - не одно и то же.

Матричный подход отвечает на вопрос: *как эффективно записать и посчитать прямой и обратный проход для целого батча?*

Autograd отвечает на вопрос: *как не писать backward вручную и автоматически получить производные?*

Поэтому матричную запись можно использовать и без autograd. В этом случае формулы градиентов мы всё ещё выводим сами, но вычисляем их сразу для целых матриц.

5.2. Градиенты в матричной форме

Для среднеквадратичной ошибки удобно записать:

$$L = \frac{1}{n} \|\hat{Y} - Y\|_F^2.$$

Тогда градиенты будут иметь вид

$$d\hat{Y} = \frac{2}{n}(\hat{Y} - Y), \quad dW = X^T d\hat{Y}, \quad db = \sum_{i=1}^n d\hat{Y}_i.$$

Это всё тот же backpropagation, но уже не по отдельным числам, а в компактной форме линейной алгебры.

5.3. Пример: ручное обучение в матричной форме без autograd

```
1 import numpy as np
2
3 X = np.array([
4     [1.0, 2.0],
5     [0.5, -1.0],
6     [3.0, 0.0],
7     [1.5, 1.0]
8 ])
9
```

```

10 Y = np.array([
11     [1.0],
12     [0.0],
13     [2.0],
14     [1.5]
15 ])
16
17 rng = np.random.default_rng(42)
18 W = rng.normal(scale=0.1, size=(2, 1))
19 b = np.zeros((1, 1))
20
21 lr = 0.1
22 n = X.shape[0]
23
24 for step in range(200):
25     # forward
26     Y_pred = X @ W + b
27     diff = Y_pred - Y
28     loss = np.mean(diff ** 2)
29
30     # backward:
31     dY = 2.0 * diff / n
32     dW = X.T @ dY
33     db = np.sum(dY, axis=0, keepdims=True)
34
35     # update
36     W -= lr * dW
37     b -= lr * db
38
39 print("loss =", loss)
40 print("W =", W.ravel())
41 print("b =", b.ravel())

```

Здесь обратный проход всё ещё написан руками, но вычисления уже организованы правильно с точки зрения производительности. Мы не перебираем элементы матрицы двойными циклами Python, а передаём работу библиотекам линейной алгебры, которые умеют считать такие операции очень быстро.

5.4. Почему матричный подход эффективен

Переход к матричной записи даёт выигрыш сразу по нескольким причинам.

Во-первых, код становится компактнее: одна строка $X @ W + b$ выражает то, что в скалярной форме заняло бы много циклов.

Во-вторых, тяжёлая работа передаётся не интерпретатору Python, а низкоуровневым библиотекам линейной алгебры, которые написаны на C/C++/Fortran и оптимизированы под конкретное железо.

В-третьих, именно матричная форма естественно переносится на GPU, где особенно хорошо работают большие однотипные операции над тензорами.

Итог: матричная алгебра не заменяет backpropagation, а делает его практичным и быстрым.

6. Autograd в PyTorch: тот же backpropagation, но уже автоматически

Теперь можно соединить две идеи вместе.

С одной стороны, нам нужен backpropagation как математический механизм вычисления производных по правилу цепочки. С другой стороны, нам нужна матричная форма, чтобы считать всё быстро. PyTorch объединяет это в единую систему: мы пишем обычный код на тензорах, а autograd сам строит граф операций и потом автоматически считает градиенты.

6.1. Тот же матричный пример, но уже с autograd

Ниже показан тот же линейный слой, что и в предыдущем разделе, но теперь формулы для dW и db мы не выписываем вручную.

```
1 import torch
2
3 X = torch.tensor([
4     [1.0, 2.0],
5     [0.5, -1.0],
6     [3.0, 0.0],
7     [1.5, 1.0]
8 ])
9
10 Y = torch.tensor([
11     [1.0],
12     [0.0],
13     [2.0],
14     [1.5]
15 ])
16
17 W = torch.randn(2, 1, requires_grad=True)
18 b = torch.zeros(1, requires_grad=True)
19
20 lr = 0.1
21
22 for step in range(200):
23     # forward
24     Y_pred = X @ W + b
25     loss = ((Y_pred - Y) ** 2).mean()
26
27     # backward:
28     loss.backward()
29
30     # update:
31     with torch.no_grad():
32         W -= lr * W.grad
```

```

33         b -= lr * b.grad
34
35     # PyTorch ,
36     W.grad.zero_()
37     b.grad.zero_()
38
39 print("loss =", loss.item())
40 print("W =", W.detach().view(-1))
41 print("b =", b.detach())

```

6.2. Что здесь делает autograd

Во время вычисления выражения

```
loss = ((X @ W + b - Y) ** 2).mean()
```

PyTorch строит граф из элементарных операций:

- матричное умножение;
- сложение;
- вычитание;
- возведение в квадрат;
- усреднение.

Когда вызывается `loss.backward()`, система идёт по этому графу в обратном порядке и на каждом шаге применяет локальную производную. Результат этого обхода записывается в `W.grad` и `b.grad`.

То есть математически это всё тот же backpropagation. Разница лишь в том, что теперь разработчику не нужно вручную реализовывать каждый шаг обратного прохода.

6.3. Две детали, которые почти всегда всплывают в начале

Почему нужен `with torch.no_grad()`? Потому что обновление параметров - это техническая операция оптимизации, а не часть модели. Мы не хотим, чтобы PyTorch строил граф и для шага обновления весов.

Почему нужно обнулять градиенты? Потому что в PyTorch градиенты **накапливаются**. Если не делать `zero_()` или `optimizer.zero_grad()`, на следующей итерации к старому градиенту добавится новый.

7. Нормальный рабочий вариант: `torch.optim`

На практике обновлять параметры вручную обычно не нужно. После того как autograd вычислил градиенты, обновление параметров передают объекту-оптимизатору из `torch.optim`. Это и есть наиболее удобный уровень абстракции для повседневной работы.

Важно понимать разделение ролей:

- autograd вычисляет градиенты;
- torch.optim использует эти градиенты, чтобы обновить параметры.

7.1. Пример небольшой модели с Adam

```

1 import torch
2 import torch.nn as nn
3
4 torch.manual_seed(42)
5
6 #
7 X = torch.linspace(-2, 2, 200).unsqueeze(1)
8 Y = 3 * X - 1 + 0.3 * torch.sin(3 * X)
9
10 #
11 model = nn.Sequential(
12     nn.Linear(1, 16),
13     nn.Tanh(),
14     nn.Linear(16, 1)
15 )
16
17 criterion = nn.MSELoss()
18 optimizer = torch.optim.Adam(model.parameters(), lr=0.03)
19
20 for epoch in range(1000):
21     # forward
22     pred = model(X)
23     loss = criterion(pred, Y)
24
25     # backward
26     optimizer.zero_grad()
27     loss.backward()
28
29     # update
30     optimizer.step()
31
32     if epoch % 200 == 0:
33         print(f"epoch={epoch:4d} loss={loss.item():.6f}")

```

В этом примере уже почти не видно механики вычисления производных, но она никуда не исчезла. Просто разные уровни системы разделены между собой:

1. модель делает прямой проход;
2. функция потерь оценивает ошибку;
3. autograd считает градиенты;
4. оптимизатор обновляет параметры.

Именно поэтому связка `nn.Module + autograd + torch.optim` считается естественной “вершиной” эволюции этого процесса: математика остаётся прежней, но большая часть рутинной работы уходит в библиотеку.

8. Итоговая картина

Теперь можно коротко сформулировать, как устроена эта эволюция.

Этап	Что делаем руками	Главное ограничение
Наивный градиентный спуск	Выводим производную и обновляем параметры	Работает только на совсем маленьких примерах
Символьное дифференцирование	Задаём формулу, система строит формулу производной	Формулы разрастаются и плохо подходят для больших тензорных вычислений
Ручной backpropagation	Разбиваем производную на локальные шаги	Нужно вручную писать backward для каждого вычисления
Матричный подход	Пишем градиенты в форме линейной алгебры	Формулы всё ещё надо выводить самим
Autograd	Пишем только forward, backward строится автоматически	Нужно понимать, где граф создаётся и как живут градиенты
<code>torch.optim</code>	Передаём обновление параметров оптимизатору	Требуется понимания, что оптимизатор не заменяет autograd